

Booking System Integration Guide

For Barbershops & Nail Salons (Replit Apps)

Version: 1.8 Date: January 2026

READ THIS FIRST

Your Current Setup

You have **two Replit apps** that serve **two business types**:

App	Purpose	Users
Consumer App	Book appointments, view bookings	Customers
Merchant App	Manage business, staff, services, calendar	Business owners

Two Business Types (Same App, Different Experience)

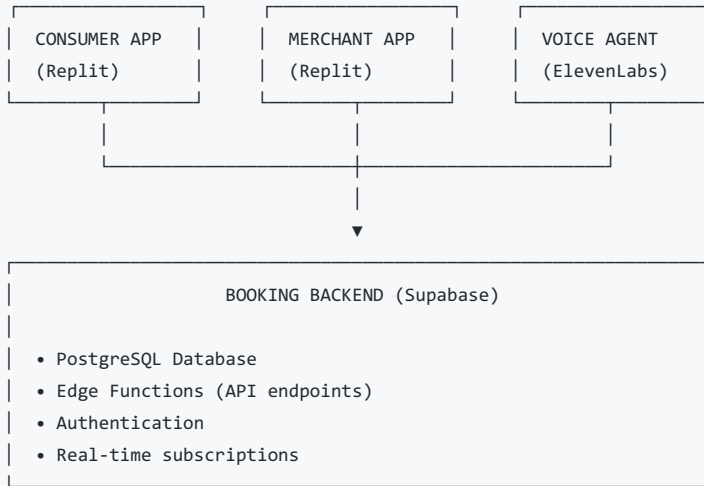
	Barbershops	Nail Salons
Staff Title	Barbers	Nail Technicians
Example Services	Haircut, Fade, Beard Trim, Shave	Manicure, Pedicure, Gel Nails, Nail Art
Service Categories	Haircut, Shave, Color, Styling	Manicure, Pedicure, Enhancements, Art
Typical Duration	15-60 min	30-120 min
Voice Agent Tone	Casual, friendly	Professional, relaxing

IMPORTANT: The system stores a `business_type` field ('barbershop' or 'nail_salon') that controls:

- Default service templates
- Staff terminology in UI and voice agent
- Service categories
- Voice agent personality/script

What We're Adding

A **booking backend** powered by Supabase that both apps connect to:



Build Order

PHASE 1: Backend Setup

- └─ Create Supabase project (or use existing)
- └─ Run database schema SQL
- └─ Deploy Edge Functions
- └─ Set up cron job for calendar sync

PHASE 2: Connect Replit Apps to Supabase

- └─ Install @supabase/supabase-js in both apps
- └─ Set environment variables (Secrets in Replit)
- └─ Create Supabase client utility

PHASE 3: Merchant App Features

- └─ Staff management (CRUD)
- └─ Services management (CRUD)
- └─ Working hours configuration
- └─ Calendar view
- └─ Calendar sync settings
- └─ API key generation

PHASE 4: Consumer App Features

- └─ Service selection
- └─ Staff selection
- └─ Date/time picker with availability
- └─ Booking confirmation
- └─ My Bookings page
- └─ Cancel booking

PHASE 5: Voice Agent

- └─ Create ElevenLabs agent
- └─ Configure tools
- └─ Connect phone number
- └─ Test end-to-end

Table of Contents

1. [Supabase Setup](#)
2. [Database Schema](#)
3. [Connecting Replit to Supabase](#)
4. [API Functions \(Edge Functions\)](#)
5. [Merchant App Code](#)
6. [Consumer App Code](#)
7. [ElevenLabs Voice Agent](#)
8. [Calendar Sync](#)

9. [SMS Notifications](#)
10. [Testing & Troubleshooting](#)

1. Supabase Setup

Step 1.1: Create Supabase Project

1. Go to supabase.com
2. Click "New Project"
3. Fill in:
 - **Name:** barbershop-booking
 - **Database Password:** Generate and SAVE this
 - **Region:** Closest to your users
4. Wait 2-3 minutes

Step 1.2: Get Your Credentials

After project is created, go to **Settings → API** and copy:

```
Project URL:      https://abcdefgh.supabase.co
Anon (public) Key: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Service Role Key: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... (KEEP SECRET!)
```

IMPORTANT:

- Anon Key = Safe for frontend (Consumer & Merchant apps)
- Service Role Key = Secret, only for backend/Edge Functions

2. Database Schema

Step 2.1: Run This SQL

Go to Supabase → **SQL Editor** → **New Query** → Paste this entire block → Click **Run**:

```
-- =====
-- BOOKING SYSTEM DATABASE SCHEMA
-- =====

-- Businesses (barbershops and nail salons)
CREATE TABLE IF NOT EXISTS businesses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  owner_user_id UUID REFERENCES auth.users(id),

  -- IMPORTANT: Determines the business type
  business_type VARCHAR(20) NOT NULL DEFAULT 'barbershop', -- 'barbershop' or 'nail_salon'

  name VARCHAR(255) NOT NULL,
  slug VARCHAR(100) UNIQUE NOT NULL,
  phone VARCHAR(20),
  email VARCHAR(255) NOT NULL,
  timezone VARCHAR(50) DEFAULT 'America/New_York',

  address_line1 VARCHAR(255),
  address_line2 VARCHAR(255),
  city VARCHAR(100),
  state VARCHAR(50),
  zip_code VARCHAR(20),
  country VARCHAR(50) DEFAULT 'US',

  -- Booking settings
```

```

booking_buffer_minutes INTEGER DEFAULT 15,
max_advance_booking_days INTEGER DEFAULT 30,
min_advance_booking_hours INTEGER DEFAULT 1,
cancellation_policy_hours INTEGER DEFAULT 24,

-- Voice agent
elevenlabs_agent_id VARCHAR(100),
business_description TEXT,

is_active BOOLEAN DEFAULT true,

-- Constraint to ensure valid business type
CONSTRAINT valid_business_type CHECK (business_type IN ('barbershop', 'nail_salon'))
);

-- Staff members (barbers for barbershops, nail technicians for nail salons)
CREATE TABLE IF NOT EXISTS staff_members (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),

    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    display_name VARCHAR(200),
    email VARCHAR(255),
    phone VARCHAR(20),
    photo_url TEXT,

    -- Role/title (auto-set based on business_type, but can be customized)
    -- Barbershop: 'barber', 'master_barber', 'apprentice'
    -- Nail Salon: 'nail_tech', 'senior_tech', 'nail_artist'
    role VARCHAR(50),

    is_active BOOLEAN DEFAULT true,
    accepts_new_clients BOOLEAN DEFAULT true,
    bio TEXT,
    specialties TEXT[], -- e.g., ['fades', 'beards'] or ['gel', 'nail art', 'acrylics']

    UNIQUE(business_id, email)
);

-- Services offered
-- Categories vary by business type:
-- Barbershop: 'haircut', 'shave', 'color', 'styling', 'treatment', 'combo'
-- Nail Salon: 'manicure', 'pedicure', 'enhancements', 'nail_art', 'treatment', 'combo'
CREATE TABLE IF NOT EXISTS services (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),

    name VARCHAR(200) NOT NULL,
    description TEXT,
    duration_minutes INTEGER NOT NULL,
    buffer_minutes INTEGER DEFAULT 0,

    price_cents INTEGER NOT NULL, -- 3500 = $35.00
    price_type VARCHAR(20) DEFAULT 'fixed', -- 'fixed', 'starting_at', 'varies'

    is_active BOOLEAN DEFAULT true,
    display_order INTEGER DEFAULT 0,
    category VARCHAR(100), -- See categories above based on business_type
    image_url TEXT,

```

```

-- For voice agent
spoken_name VARCHAR(200),
spoken_description TEXT
);

-- =====
-- DEFAULT SERVICE TEMPLATES
-- Run these AFTER creating a business to populate default services
-- =====

-- Function to create default services for a new business
CREATE OR REPLACE FUNCTION create_default_services(p_business_id UUID, p_business_type VARCHAR)
RETURNS void AS $$
BEGIN
    IF p_business_type = 'barbershop' THEN
        -- Barbershop default services
        INSERT INTO services (business_id, name, duration_minutes, price_cents, category, display_order, spoken_name) VALUES
            (p_business_id, 'Classic Haircut', 30, 3500, 'haircut', 1, 'classic haircut'),
            (p_business_id, 'Fade', 30, 4000, 'haircut', 2, 'fade haircut'),
            (p_business_id, 'Buzz Cut', 20, 2500, 'haircut', 3, 'buzz cut'),
            (p_business_id, 'Kids Haircut', 25, 2500, 'haircut', 4, 'kids haircut'),
            (p_business_id, 'Beard Trim', 15, 1500, 'shave', 5, 'beard trim'),
            (p_business_id, 'Hot Towel Shave', 30, 3000, 'shave', 6, 'hot towel shave'),
            (p_business_id, 'Haircut & Beard Combo', 45, 5000, 'combo', 7, 'haircut and beard combo'),
            (p_business_id, 'Hair Color', 60, 6000, 'color', 8, 'hair coloring');

    ELSIF p_business_type = 'nail_salon' THEN
        -- Nail salon default services
        INSERT INTO services (business_id, name, duration_minutes, price_cents, category, display_order, spoken_name) VALUES
            (p_business_id, 'Basic Manicure', 30, 2500, 'manicure', 1, 'basic manicure'),
            (p_business_id, 'Gel Manicure', 45, 4000, 'manicure', 2, 'gel manicure'),
            (p_business_id, 'Basic Pedicure', 45, 3500, 'pedicure', 3, 'basic pedicure'),
            (p_business_id, 'Spa Pedicure', 60, 5500, 'pedicure', 4, 'spa pedicure'),
            (p_business_id, 'Mani-Pedi Combo', 75, 5500, 'combo', 5, 'mani pedi combo'),
            (p_business_id, 'Acrylic Full Set', 60, 5000, 'enhancements', 6, 'acrylic full set'),
            (p_business_id, 'Acrylic Fill', 45, 3500, 'enhancements', 7, 'acrylic fill'),
            (p_business_id, 'Gel Extensions', 90, 7500, 'enhancements', 8, 'gel extensions'),
            (p_business_id, 'Nail Art (per nail)', 10, 500, 'nail_art', 9, 'nail art'),
            (p_business_id, 'Polish Change', 15, 1500, 'manicure', 10, 'polish change');
    END IF;
END;
$$ LANGUAGE plpgsql;

-- Which staff can do which services
CREATE TABLE IF NOT EXISTS staff_services (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    staff_id UUID REFERENCES staff_members(id) ON DELETE CASCADE,
    service_id UUID REFERENCES services(id) ON DELETE CASCADE,
    custom_duration_minutes INTEGER,
    custom_price_cents INTEGER,
    is_active BOOLEAN DEFAULT true,
    UNIQUE(staff_id, service_id)
);

-- Weekly working hours
CREATE TABLE IF NOT EXISTS working_hours (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    staff_id UUID REFERENCES staff_members(id) ON DELETE CASCADE,
    day_of_week INTEGER NOT NULL CHECK (day_of_week >= 0 AND day_of_week <= 6),
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    is_active BOOLEAN DEFAULT true,
    UNIQUE(staff_id, day_of_week)
);

-- Blocked times (from external calendars or manual)

```

```

CREATE TABLE IF NOT EXISTS blocked_times (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,
  staff_id UUID REFERENCES staff_members(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ DEFAULT NOW(),

  start_time TIMESTAMPTZ NOT NULL,
  end_time TIMESTAMPTZ NOT NULL,

  source VARCHAR(50) NOT NULL, -- 'manual', 'vagaro', 'booksy', 'google', 'ical'
  external_id VARCHAR(255),
  title VARCHAR(255),
  reason VARCHAR(50)
);

CREATE INDEX IF NOT EXISTS idx_blocked_times_lookup
  ON blocked_times(business_id, staff_id, start_time, end_time);

-- Bookings (appointments)
CREATE TABLE IF NOT EXISTS bookings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,
  staff_id UUID REFERENCES staff_members(id) ON DELETE SET NULL,
  service_id UUID REFERENCES services(id) ON DELETE SET NULL,
  client_id UUID,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),

  start_time TIMESTAMPTZ NOT NULL,
  end_time TIMESTAMPTZ NOT NULL,
  duration_minutes INTEGER NOT NULL,

  client_name VARCHAR(200) NOT NULL,
  client_phone VARCHAR(20) NOT NULL,
  client_email VARCHAR(255),

  status VARCHAR(50) DEFAULT 'confirmed',
  cancelled_at TIMESTAMPTZ,
  cancellation_reason TEXT,

  price_cents INTEGER NOT NULL,
  booking_source VARCHAR(50) DEFAULT 'consumer_app',
  voice_call_id VARCHAR(255),

  client_notes TEXT,
  staff_notes TEXT,

  confirmation_number VARCHAR(20) UNIQUE,
  confirmation_sent_at TIMESTAMPTZ,
  reminder_sent_at TIMESTAMPTZ
);

CREATE INDEX IF NOT EXISTS idx_bookings_business ON bookings(business_id, start_time);
CREATE INDEX IF NOT EXISTS idx_bookings_staff ON bookings(staff_id, start_time);
CREATE INDEX IF NOT EXISTS idx_bookings_phone ON bookings(client_phone);

-- Calendar sync configurations
CREATE TABLE IF NOT EXISTS calendar_syncs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,
  staff_id UUID REFERENCES staff_members(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),

  provider VARCHAR(50) NOT NULL,

```

```

provider_name VARCHAR(100),
ical_url TEXT,

last_synced_at TIMESTAMPTZ,
last_sync_status VARCHAR(50),
last_sync_error TEXT,
events_count INTEGER DEFAULT 0,

is_active BOOLEAN DEFAULT true,
UNIQUE(business_id, staff_id, provider)
);

-- Clients (customers)
CREATE TABLE IF NOT EXISTS clients (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,
    user_id UUID REFERENCES auth.users(id),
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),

    phone VARCHAR(20) NOT NULL,
    email VARCHAR(255),
    first_name VARCHAR(100),
    last_name VARCHAR(100),

    preferred_staff_id UUID REFERENCES staff_members(id),
    notes TEXT,

    total_bookings INTEGER DEFAULT 0,
    total_spent_cents INTEGER DEFAULT 0,
    last_visit_at TIMESTAMPTZ,

    UNIQUE(business_id, phone)
);

-- API keys for voice agent
CREATE TABLE IF NOT EXISTS api_keys (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,
    key_hash VARCHAR(64) NOT NULL,
    key_prefix VARCHAR(16) NOT NULL,
    name VARCHAR(100) DEFAULT 'Voice Agent Key',
    created_at TIMESTAMPTZ DEFAULT NOW(),
    last_used_at TIMESTAMPTZ,
    is_active BOOLEAN DEFAULT true
);

-- =====
-- FUNCTIONS & TRIGGERS
-- =====

-- Auto-generate confirmation numbers
CREATE OR REPLACE FUNCTION generate_confirmation_number()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.confirmation_number IS NULL THEN
        NEW.confirmation_number := 'BK-' || LPAD(FLOOR(RANDOM() * 10000)::TEXT, 4, '0');
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS set_confirmation_number ON bookings;
CREATE TRIGGER set_confirmation_number
BEFORE INSERT ON bookings FOR EACH ROW
EXECUTE FUNCTION generate_confirmation_number();

```

```

EXECUTE FUNCTION generate_confirmation_number();

-- Auto-update timestamps
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_businesses_timestamp BEFORE UPDATE ON businesses
FOR EACH ROW EXECUTE FUNCTION update_updated_at();
CREATE TRIGGER update_staff_timestamp BEFORE UPDATE ON staff_members
FOR EACH ROW EXECUTE FUNCTION update_updated_at();
CREATE TRIGGER update_services_timestamp BEFORE UPDATE ON services
FOR EACH ROW EXECUTE FUNCTION update_updated_at();
CREATE TRIGGER update_bookings_timestamp BEFORE UPDATE ON bookings
FOR EACH ROW EXECUTE FUNCTION update_updated_at();

-- =====
-- ROW LEVEL SECURITY
-- =====

ALTER TABLE businesses ENABLE ROW LEVEL SECURITY;
ALTER TABLE staff_members ENABLE ROW LEVEL SECURITY;
ALTER TABLE services ENABLE ROW LEVEL SECURITY;
ALTER TABLE staff_services ENABLE ROW LEVEL SECURITY;
ALTER TABLE working_hours ENABLE ROW LEVEL SECURITY;
ALTER TABLE blocked_times ENABLE ROW LEVEL SECURITY;
ALTER TABLE bookings ENABLE ROW LEVEL SECURITY;
ALTER TABLE calendar_syncs ENABLE ROW LEVEL SECURITY;
ALTER TABLE clients ENABLE ROW LEVEL SECURITY;
ALTER TABLE api_keys ENABLE ROW LEVEL SECURITY;

-- Merchant policies (full access to own business)
CREATE POLICY "Owners manage own business" ON businesses
FOR ALL USING (auth.uid() = owner_user_id);

CREATE POLICY "Owners manage own staff" ON staff_members
FOR ALL USING (business_id IN (SELECT id FROM businesses WHERE owner_user_id = auth.uid()));

CREATE POLICY "Owners manage own services" ON services
FOR ALL USING (business_id IN (SELECT id FROM businesses WHERE owner_user_id = auth.uid()));

CREATE POLICY "Owners manage staff_services" ON staff_services
FOR ALL USING (staff_id IN (
    SELECT id FROM staff_members WHERE business_id IN (
        SELECT id FROM businesses WHERE owner_user_id = auth.uid()
    )
));

CREATE POLICY "Owners manage working_hours" ON working_hours
FOR ALL USING (staff_id IN (
    SELECT id FROM staff_members WHERE business_id IN (
        SELECT id FROM businesses WHERE owner_user_id = auth.uid()
    )
));

CREATE POLICY "Owners manage blocked_times" ON blocked_times
FOR ALL USING (business_id IN (SELECT id FROM businesses WHERE owner_user_id = auth.uid()));

CREATE POLICY "Owners manage bookings" ON bookings
FOR ALL USING (business_id IN (SELECT id FROM businesses WHERE owner_user_id = auth.uid()));

CREATE POLICY "Owners manage calendar_syncs" ON calendar_syncs

```



```

FOR ALL USING (business_id IN (SELECT id FROM businesses WHERE owner_user_id = auth.uid()));

CREATE POLICY "Owners manage clients" ON clients
  FOR ALL USING (business_id IN (SELECT id FROM businesses WHERE owner_user_id = auth.uid()));

CREATE POLICY "Owners manage api_keys" ON api_keys
  FOR ALL USING (business_id IN (SELECT id FROM businesses WHERE owner_user_id = auth.uid()));

-- Consumer policies (read active businesses/staff/services, manage own bookings)
CREATE POLICY "Public read active businesses" ON businesses
  FOR SELECT USING (is_active = true);

CREATE POLICY "Public read active staff" ON staff_members
  FOR SELECT USING (is_active = true);

CREATE POLICY "Public read active services" ON services
  FOR SELECT USING (is_active = true);

CREATE POLICY "Public read staff_services" ON staff_services
  FOR SELECT USING (is_active = true);

CREATE POLICY "Public read working_hours" ON working_hours
  FOR SELECT USING (is_active = true);

CREATE POLICY "Authenticated create bookings" ON bookings
  FOR INSERT WITH CHECK (auth.uid() IS NOT NULL);

CREATE POLICY "Users view own bookings" ON bookings
  FOR SELECT USING (
    client_id IN (SELECT id FROM clients WHERE user_id = auth.uid())
  );

CREATE POLICY "Users update own bookings" ON bookings
  FOR UPDATE USING (
    client_id IN (SELECT id FROM clients WHERE user_id = auth.uid())
  );

CREATE POLICY "Users manage own client record" ON clients
  FOR ALL USING (user_id = auth.uid());

-- =====
-- DONE! Verify with:
-- SELECT table_name FROM information_schema.tables WHERE table_schema = 'public';
-- =====

```

3. Connecting Replit to Supabase

Step 3.1: Install Supabase in Both Replit Projects

In both your **Consumer App** and **Merchant App** Replit projects:

If using **npm** (Node.js/React/Next.js):

```
npm install @supabase/supabase-js
```

If using **Python**:

```
pip install supabase
```

Step 3.2: Add Environment Variables (Secrets)

In Replit, go to **Tools** → **Secrets** and add:

Key	Value	Used In
SUPABASE_URL	https://xxx.supabase.co	Both apps
SUPABASE_ANON_KEY	eyJ... (public key)	Both apps
SUPABASE_SERVICE_ROLE_KEY	eyJ... (secret key)	Backend only
TWILIO_ACCOUNT_SID	From Twilio	Backend only
TWILIO_AUTH_TOKEN	From Twilio	Backend only
TWILIO_PHONE_NUMBER	+1...	Backend only

Step 3.3: Create Supabase Client

JavaScript/TypeScript (React, Next.js, etc.)

Create a file `lib/supabase.js` (or `.ts`):

```
// lib/supabase.js

import { createClient } from '@supabase/supabase-js';

const supabaseUrl = process.env.SUPABASE_URL || process.env.NEXT_PUBLIC_SUPABASE_URL;
const supabaseAnonKey = process.env.SUPABASE_ANON_KEY || process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

if (!supabaseUrl || !supabaseAnonKey) {
  throw new Error('Missing Supabase environment variables');
}

export const supabase = createClient(supabaseUrl, supabaseAnonKey);

// For server-side operations that need elevated privileges
export const createServiceClient = () => {
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;
  if (!serviceKey) {
    throw new Error('Missing SUPABASE_SERVICE_ROLE_KEY');
  }
  return createClient(supabaseUrl, serviceKey);
};
```

Python (Flask, FastAPI, etc.)

Create a file `lib/supabase.py`:

```
# lib/supabase.py

import os
from supabase import create_client, Client

SUPABASE_URL = os.environ.get('SUPABASE_URL')
SUPABASE_ANON_KEY = os.environ.get('SUPABASE_ANON_KEY')
SUPABASE_SERVICE_KEY = os.environ.get('SUPABASE_SERVICE_ROLE_KEY')

if not SUPABASE_URL or not SUPABASE_ANON_KEY:
    raise ValueError('Missing Supabase environment variables')

# For client-side operations
supabase: Client = create_client(SUPABASE_URL, SUPABASE_ANON_KEY)

# For server-side operations
def get_service_client() -> Client:
    if not SUPABASE_SERVICE_KEY:
        raise ValueError('Missing SUPABASE_SERVICE_ROLE_KEY')
    return create_client(SUPABASE_URL, SUPABASE_SERVICE_KEY)
```

4. API Functions (Edge Functions)

These run on Supabase's servers and handle the core booking logic.

Step 4.1: Install Supabase CLI

On your local machine (not Replit):

```
npm install -g supabase
supabase login
supabase link --project-ref YOUR_PROJECT_REF
```

Step 4.2: Create `get-availability` Function

Create file: `supabase/functions/get-availability/index.ts`

```
import { serve } from "https://deno.land/std@0.168.0/http/server.ts"
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2'

const corsHeaders = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type, x-api-key',
}

serve(async (req) => {
  if (req.method === 'OPTIONS') {
    return new Response('ok', { headers: corsHeaders })
  }

  try {
    const url = new URL(req.url)
    const business_id = url.searchParams.get('business_id')
    const service_id = url.searchParams.get('service_id')
    const staff_id = url.searchParams.get('staff_id')
    const date = url.searchParams.get('date') // YYYY-MM-DD

    if (!business_id || !service_id || !date) {
      return new Response(
        JSON.stringify({ error: 'Missing: business_id, service_id, date' }),
        { status: 400, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
      )
    }
  } catch (error) {
    return new Response(
      JSON.stringify({ error: 'Internal Server Error' }),
      { status: 500, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
    )
  }
})
```

```

    )
  }

  const supabase = createClient(
    Deno.env.get('SUPABASE_URL')!,
    Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!
  )

  // Get service
  const { data: service } = await supabase
    .from('services')
    .select('*')
    .eq('id', service_id)
    .single()

  if (!service) {
    return new Response(
      JSON.stringify({ error: 'Service not found' }),
      { status: 404, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
    )
  }

  // Get business settings
  const { data: business } = await supabase
    .from('businesses')
    .select('*')
    .eq('id', business_id)
    .single()

  if (!business) {
    return new Response(
      JSON.stringify({ error: 'Business not found' }),
      { status: 404, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
    )
  }

  // Get staff who can do this service
  const { data: staffServices } = await supabase
    .from('staff_services')
    .select(`
      staff_id,
      staff_members!inner(id, first_name, last_name, display_name, photo_url)
    `)
    .eq('service_id', service_id)
    .eq('is_active', true)

  let eligibleStaff = (staffServices || []).map(ss => ({
    id: ss.staff_members.id,
    name: ss.staff_members.display_name || ss.staff_members.first_name,
    photo_url: ss.staff_members.photo_url
  })))

  if (staff_id) {
    eligibleStaff = eligibleStaff.filter(s => s.id === staff_id)
  }

  if (eligibleStaff.length === 0) {
    return new Response(
      JSON.stringify({ available_slots: [], message: 'No staff available' }),
      { status: 200, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
    )
  }

  const dayOfWeek = new Date(date + 'T12:00:00').getDay()
  const totalDuration = service.duration_minutes + (service.buffer_minutes || 0)

```

```

const allSlots: any[] = []

for (const staff of eligibleStaff) {
  // Get working hours
  const { data: hours } = await supabase
    .from('working_hours')
    .select('start_time, end_time')
    .eq('staff_id', staff.id)
    .eq('day_of_week', dayOfWeek)
    .eq('is_active', true)
    .single()

  if (!hours) continue

  // Get existing bookings
  const { data: bookings } = await supabase
    .from('bookings')
    .select('start_time, end_time')
    .eq('staff_id', staff.id)
    .in('status', ['pending', 'confirmed'])
    .gte('start_time', date + 'T00:00:00')
    .lte('start_time', date + 'T23:59:59')

  // Get blocked times
  const { data: blocked } = await supabase
    .from('blocked_times')
    .select('start_time, end_time')
    .or(`staff_id.eq.${staff.id},staff_id.is.null`)
    .eq('business_id', business_id)
    .gte('start_time', date + 'T00:00:00')
    .lte('start_time', date + 'T23:59:59')

  const busyTimes = [...(bookings || []), ...(blocked || [])]

  // Generate slots
  const slots = generateSlots(
    date,
    hours.start_time,
    hours.end_time,
    totalDuration,
    busyTimes,
    business.booking_buffer_minutes || 0,
    business.min_advance_booking_hours || 1
  )

  for (const slot of slots) {
    allSlots.push({
      staff_id: staff.id,
      staff_name: staff.name,
      staff_photo: staff.photo_url,
      start_time: slot.start,
      end_time: slot.end,
      formatted_time: formatTime(new Date(slot.start))
    })
  }
}

allSlots.sort((a, b) => new Date(a.start_time).getTime() - new Date(b.start_time).getTime())

return new Response(
  JSON.stringify({
    business_id,
    date,
    service: {
      id: service.id,
      name: service.name,

```

```

        duration_minutes: service.duration_minutes,
        price_cents: service.price_cents,
        price_formatted: '$' + (service.price_cents / 100).toFixed(2)
      },
      available_slots: allSlots
    )),
    { status: 200, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
  )
} catch (error) {
  console.error('Error:', error)
  return new Response(
    JSON.stringify({ error: 'Server error' }),
    { status: 500, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
  )
}
})

function generateSlots(date: string, startTime: string, endTime: string,
  duration: number, busyTimes: any[], buffer: number, minAdvance: number) {

  const slots: { start: string; end: string }[] = []
  const workStart = new Date(`${date}T${startTime}`)
  const workEnd = new Date(`${date}T${endTime}`)
  const earliest = new Date(Date.now() + minAdvance * 60 * 60 * 1000)

  let current = new Date(workStart)

  while (current < workEnd) {
    const slotEnd = new Date(current.getTime() + duration * 60 * 1000)

    if (slotEnd > workEnd) break
    if (current < earliest) {
      current = new Date(current.getTime() + 15 * 60 * 1000)
      continue
    }

    const hasConflict = busyTimes.some(busy => {
      const busyStart = new Date(busy.start_time)
      const busyEnd = new Date(busy.end_time)
      const bufferedStart = new Date(busyStart.getTime() - buffer * 60 * 1000)
      const bufferedEnd = new Date(busyEnd.getTime() + buffer * 60 * 1000)
      return current < bufferedEnd && slotEnd > bufferedStart
    })

    if (!hasConflict) {
      slots.push({ start: current.toISOString(), end: slotEnd.toISOString() })
    }

    current = new Date(current.getTime() + 15 * 60 * 1000)
  }

  return slots
}

function formatTime(date: Date): string {
  return date.toLocaleTimeString('en-US', { hour: 'numeric', minute: '2-digit', hour12: true })
}

```

Step 4.3: Create `create-booking` Function

Create file: `supabase/functions/create-booking/index.ts`

```
import { serve } from "https://deno.land/std@0.168.0/http/server.ts"
```

```

import { createClient } from 'https://esm.sh/@supabase/supabase-js@2'

const corsHeaders = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
}

serve(async (req) => {
  if (req.method === 'OPTIONS') {
    return new Response('ok', { headers: corsHeaders })
  }

  try {
    const supabase = createClient(
      Deno.env.get('SUPABASE_URL')!,
      Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!
    )

    const body = await req.json()
    const {
      business_id, service_id, staff_id, start_time,
      client_name, client_phone, client_email, client_notes,
      user_id, booking_source = 'consumer_app'
    } = body

    if (!business_id || !service_id || !staff_id || !start_time || !client_name || !client_phone) {
      return new Response(
        JSON.stringify({ success: false, error: 'Missing required fields' }),
        { status: 400, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
      )
    }

    // Get service
    const { data: service } = await supabase
      .from('services')
      .select('duration_minutes, price_cents, name')
      .eq('id', service_id)
      .single()

    if (!service) {
      return new Response(
        JSON.stringify({ success: false, error: 'Service not found' }),
        { status: 404, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
      )
    }

    const startDT = new Date(start_time)
    const endDT = new Date(startDT.getTime() + service.duration_minutes * 60 * 1000)

    // Check for conflicts
    const { data: conflicts } = await supabase
      .from('bookings')
      .select('id')
      .eq('staff_id', staff_id)
      .in('status', ['pending', 'confirmed'])
      .lt('start_time', endDT.toISOString())
      .gt('end_time', startDT.toISOString())

    if (conflicts && conflicts.length > 0) {
      return new Response(
        JSON.stringify({
          success: false,
          error: 'slot_unavailable',
          message: 'This time slot is no longer available.'
        }),
        { status: 409, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
      )
    }
  } catch (error) {
    return new Response(
      JSON.stringify({ success: false, error: error.message }),
      { status: 500, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
    )
  }
})

```

```

    }
  }

  // Normalize phone
  const normalizedPhone = normalizePhone(client_phone)

  // Find or create client
  let clientId = null
  const { data: existingClient } = await supabase
    .from('clients')
    .select('id')
    .eq('business_id', business_id)
    .eq('phone', normalizedPhone)
    .single()

  if (existingClient) {
    clientId = existingClient.id
  } else {
    const { data: newClient } = await supabase
      .from('clients')
      .insert({
        business_id,
        phone: normalizedPhone,
        first_name: client_name.split(' ')[0],
        last_name: client_name.split(' ').slice(1).join(' '),
        email: client_email,
        user_id: user_id || null
      })
      .select('id')
      .single()
    clientId = newClient?.id
  }

  // Create booking
  const { data: booking, error: bookingError } = await supabase
    .from('bookings')
    .insert({
      business_id,
      service_id,
      staff_id,
      client_id: clientId,
      start_time: startDT.toISOString(),
      end_time: endDT.toISOString(),
      duration_minutes: service.duration_minutes,
      client_name,
      client_phone: normalizedPhone,
      client_email,
      client_notes,
      price_cents: service.price_cents,
      booking_source,
      status: 'confirmed'
    })
    .select()
    .single()

  if (bookingError) {
    console.error('Booking error:', bookingError)
    return new Response(
      JSON.stringify({ success: false, error: 'Failed to create booking' }),
      { status: 500, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
    )
  }

  // Get staff name
  const { data: staff } = await supabase

```



```

    .from('staff_members')
    .select('display_name, first_name')
    .eq('id', staff_id)
    .single()

return new Response(
  JSON.stringify({
    success: true,
    booking: {
      id: booking.id,
      confirmation_number: booking.confirmation_number,
      start_time: booking.start_time,
      end_time: booking.end_time,
      formatted_datetime: formatDateTime(startDT),
      staff_name: staff?.display_name || staff?.first_name,
      service_name: service.name,
      price_formatted: '$' + (service.price_cents / 100).toFixed(2)
    }
  }),
  { status: 201, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
)

} catch (error) {
  console.error('Error:', error)
  return new Response(
    JSON.stringify({ success: false, error: 'Server error' }),
    { status: 500, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
  )
}
})

function normalizePhone(phone: string): string {
  const digits = phone.replace(/\D/g, '')
  if (digits.length === 10) return '+1' + digits
  if (digits.length === 11 && digits.startsWith('1')) return '+' + digits
  return '+' + digits
}

function formatDateTime(date: Date): string {
  return date.toLocaleDateString('en-US', {
    weekday: 'long', month: 'long', day: 'numeric',
    hour: 'numeric', minute: '2-digit', hour12: true
  })
}

```

Step 4.4: Deploy Functions

```

supabase functions deploy get-availability
supabase functions deploy create-booking

```

5. Merchant App Code

Add these features to your existing Merchant Replit app.

5.0: Business Type Helper

Create this utility first - it provides the right terminology based on business type.

```
// lib/businessTypes.js

export const BUSINESS_TYPES = {
  barbershop: {
    type: 'barbershop',
    name: 'Barbershop',
    staffSingular: 'Barber',
    staffPlural: 'Barbers',
    staffRoles: [
      { value: 'barber', label: 'Barber' },
      { value: 'master_barber', label: 'Master Barber' },
      { value: 'apprentice', label: 'Apprentice' }
    ],
    serviceCategories: [
      { value: 'haircut', label: 'Haircut' },
      { value: 'shave', label: 'Shave & Beard' },
      { value: 'color', label: 'Color' },
      { value: 'styling', label: 'Styling' },
      { value: 'treatment', label: 'Treatment' },
      { value: 'combo', label: 'Combo Package' }
    ],
    defaultSpecialties: ['Fades', 'Beard Work', 'Hot Towel Shave', 'Hair Design', 'Kids Cuts']
  },
  nail_salon: {
    type: 'nail_salon',
    name: 'Nail Salon',
    staffSingular: 'Nail Technician',
    staffPlural: 'Nail Technicians',
    staffRoles: [
      { value: 'nail_tech', label: 'Nail Technician' },
      { value: 'senior_tech', label: 'Senior Technician' },
      { value: 'nail_artist', label: 'Nail Artist' }
    ],
    serviceCategories: [
      { value: 'manicure', label: 'Manicure' },
      { value: 'pedicure', label: 'Pedicure' },
      { value: 'enhancements', label: 'Enhancements' },
      { value: 'nail_art', label: 'Nail Art' },
      { value: 'treatment', label: 'Treatment' },
      { value: 'combo', label: 'Combo Package' }
    ],
    defaultSpecialties: ['Gel', 'Acrylics', 'Nail Art', 'Dip Powder', 'French Tips', 'Pedicure']
  }
};

export function getBusinessConfig(businessType) {
  return BUSINESS_TYPES[businessType] || BUSINESS_TYPES.barbershop;
}
```

5.1: Staff Management (CRUD)

JavaScript/React Example

```
// components/StaffManagement.jsx

import { useState, useEffect } from 'react';
import { supabase } from '../lib/supabase';
import { getBusinessConfig } from '../lib/businessTypes';

export function StaffManagement({ businessId, businessType }) {
  const [staff, setStaff] = useState([]);
  const [loading, setLoading] = useState(true);
  const [showForm, setShowForm] = useState(false);
  const [editingStaff, setEditingStaff] = useState(null);
```

```

const [editingStaff, setEditingStaff] = useState(null);

// Get terminology based on business type
const config = getBusinessConfig(businessType);

// Fetch staff
useEffect(() => {
  fetchStaff();
}, [businessId]);

async function fetchStaff() {
  setLoading(true);
  const { data, error } = await supabase
    .from('staff_members')
    .select('*')
    .eq('business_id', businessId)
    .eq('is_active', true)
    .order('first_name');

  if (error) {
    console.error('Error fetching staff:', error);
  } else {
    setStaff(data || []);
  }
  setLoading(false);
}

// Create staff
async function createStaff(formData) {
  const { data, error } = await supabase
    .from('staff_members')
    .insert({
      business_id: businessId,
      first_name: formData.firstName,
      last_name: formData.lastName,
      display_name: formData.displayName || formData.firstName,
      email: formData.email,
      phone: formData.phone,
      role: formData.role,
      bio: formData.bio,
      specialties: formData.specialties,
      is_active: true,
      accepts_new_clients: true
    })
    .select()
    .single();

  if (error) {
    console.error('Error creating staff:', error);
    return { error };
  }

  setStaff([...staff, data]);
  setShowForm(false);
  return { data };
}

// Update staff
async function updateStaff(id, formData) {
  const { data, error } = await supabase
    .from('staff_members')
    .update({
      first_name: formData.firstName,
      last_name: formData.lastName,
      display_name: formData.displayName,
      email: formData.email,

```

```

        phone: formData.phone,
        role: formData.role,
        bio: formData.bio,
        specialties: formData.specialties
    })
    .eq('id', id)
    .select()
    .single();

    if (error) {
        console.error('Error updating staff:', error);
        return { error };
    }

    setStaff(staff.map(s => s.id === id ? data : s));
    setEditingStaff(null);
    return { data };
}

// Delete staff (soft delete)
async function deleteStaff(id) {
    if (!confirm(`Are you sure you want to remove this ${config.staffSingular.toLowerCase()}?`)) return;

    const { error } = await supabase
        .from('staff_members')
        .update({ is_active: false })
        .eq('id', id);

    if (error) {
        console.error('Error deleting staff:', error);
        return;
    }

    setStaff(staff.filter(s => s.id !== id));
}

// Get role label
function getRoleLabel(roleValue) {
    const role = config.staffRoles.find(r => r.value === roleValue);
    return role ? role.label : config.staffSingular;
}

// Render (implement your own UI)
return (
    <div>
        {/* Dynamic title based on business type */}
        <h2>{config.staffPlural}</h2>
        <button onClick={() => setShowForm(true)}>Add {config.staffSingular}</button>

        {loading ? (
            <p>Loading...</p>
        ) : (
            <table>
                <thead>
                    <tr>
                        <th>Name</th>
                        <th>Display Name</th>
                        <th>Role</th>
                        <th>Email</th>
                        <th>Phone</th>
                        <th>Actions</th>
                    </tr>
                </thead>
                <tbody>
                    {staff.map(s => (

```

```

        <tr key={s.id}>
          <td>{s.first_name} {s.last_name}</td>
          <td>{s.display_name}</td>
          <td>{getRoleLabel(s.role)}</td>
          <td>{s.email}</td>
          <td>{s.phone}</td>
          <td>
            <button onClick={() => setEditingStaff(s)}>Edit</button>
            <button onClick={() => deleteStaff(s.id)}>Delete</button>
          </td>
        </tr>
      </tbody>
    </table>
  )}
}

{/* Form modal - include role dropdown from config.staffRoles */}
{/* Include specialties checkboxes from config.defaultSpecialties */}
</div>
);
}

```

5.2: Services Management

```

// components/ServicesManagement.jsx

import { useState, useEffect } from 'react';
import { supabase } from '../lib/supabase';
import { getBusinessConfig } from '../lib/businessTypes';

export function ServicesManagement({ businessId, businessType }) {
  const [services, setServices] = useState([]);
  const [loading, setLoading] = useState(true);

  // Get categories based on business type
  const config = getBusinessConfig(businessType);

  useEffect(() => {
    fetchServices();
  }, [businessId]);

  async function fetchServices() {
    setLoading(true);
    const { data, error } = await supabase
      .from('services')
      .select('*')
      .eq('business_id', businessId)
      .eq('is_active', true)
      .order('display_order');

    if (!error) setServices(data || []);
    setLoading(false);
  }

  // Create default services for the business type
  async function createDefaultServices() {
    if (!confirm(`Create default ${businessType} === 'barbershop' ? 'barbershop' : 'nail salon' services?`)) return;

    // Call the database function we created
    const { error } = await supabase.rpc('create_default_services', {
      p_business_id: businessId,
      p_business_type: businessType
    });
  }
}

```

```

    if (error) {
      console.error('Error creating defaults:', error);
      alert('Error creating default services');
    } else {
      fetchServices();
    }
  }
}

async function createService(formData) {
  const { data, error } = await supabase
    .from('services')
    .insert({
      business_id: businessId,
      name: formData.name,
      description: formData.description,
      duration_minutes: parseInt(formData.duration),
      price_cents: Math.round(parseFloat(formData.price) * 100), // $35 -> 3500
      category: formData.category,
      buffer_minutes: parseInt(formData.buffer) || 0,
      spoken_name: formData.spokenName || formData.name.toLowerCase(),
      is_active: true
    })
    .select()
    .single();

  if (!error) {
    setServices([...services, data]);
  }
  return { data, error };
}

async function updateService(id, formData) {
  const { data, error } = await supabase
    .from('services')
    .update({
      name: formData.name,
      description: formData.description,
      duration_minutes: parseInt(formData.duration),
      price_cents: Math.round(parseFloat(formData.price) * 100),
      category: formData.category,
      buffer_minutes: parseInt(formData.buffer) || 0,
      spoken_name: formData.spokenName
    })
    .eq('id', id)
    .select()
    .single();

  if (!error) {
    setServices(services.map(s => s.id === id ? data : s));
  }
  return { data, error };
}

async function deleteService(id) {
  if (!confirm('Delete this service?')) return;

  const { error } = await supabase
    .from('services')
    .update({ is_active: false })
    .eq('id', id);

  if (!error) {
    setServices(services.filter(s => s.id !== id));
  }
}

```

```

// Helper to format price
function formatPrice(cents) {
  return '$' + (cents / 100).toFixed(2);
}

// Get category label
function getCategoryLabel(categoryValue) {
  const cat = config.serviceCategories.find(c => c.value === categoryValue);
  return cat ? cat.label : categoryValue;
}

// Group services by category
function getServicesByCategory() {
  const grouped = {};
  config.serviceCategories.forEach(cat => {
    grouped[cat.value] = services.filter(s => s.category === cat.value);
  });
  // Add uncategorized
  grouped['other'] = services.filter(s => !s.category);
  return grouped;
}

return (
  <div>
    <h2>Services</h2>

    {services.length === 0 && (
      <div>
        <p>No services yet.</p>
        <button onClick={createDefaultServices}>
          Create Default {businessType === 'barbershop' ? 'Barbershop' : 'Nail Salon'} Services
        </button>
      </div>
    )}

    {/* Show services grouped by category */}
    {config.serviceCategories.map(category => {
      const categoryServices = services.filter(s => s.category === category.value);
      if (categoryServices.length === 0) return null;

      return (
        <div key={category.value} className="service-category">
          <h3>{category.label}</h3>
          <div className="services-grid">
            {categoryServices.map(service => (
              <div key={service.id} className="service-card">
                <h4>{service.name}</h4>
                <p>{service.duration_minutes} min | {formatPrice(service.price_cents)}</p>
                <p>{service.description}</p>
                <button onClick={() => deleteService(service.id)}>Delete</button>
              </div>
            ))}
          </div>
        </div>
      );
    })}

    {/* Form modal - use config.serviceCategories for the category dropdown */}
  </div>
);
}

```

5.3: Working Hours

```
// components/WorkingHours.jsx

import { useState, useEffect } from 'react';
import { supabase } from '../lib/supabase';

const DAYS = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday'];

export function WorkingHours({ staffId }) {
  const [hours, setHours] = useState({});
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchHours();
  }, [staffId]);

  async function fetchHours() {
    setLoading(true);
    const { data, error } = await supabase
      .from('working_hours')
      .select('*')
      .eq('staff_id', staffId);

    if (!error) {
      // Convert array to object keyed by day_of_week
      const hoursMap = {};
      (data || []).forEach(h => {
        hoursMap[h.day_of_week] = {
          id: h.id,
          enabled: h.is_active,
          start: h.start_time,
          end: h.end_time
        };
      });
      setHours(hoursMap);
    }
    setLoading(false);
  }

  async function saveDay(dayOfWeek, enabled, startTime, endTime) {
    const existing = hours[dayOfWeek];

    if (enabled) {
      if (existing?.id) {
        // Update
        await supabase
          .from('working_hours')
          .update({
            start_time: startTime,
            end_time: endTime,
            is_active: true
          })
          .eq('id', existing.id);
      } else {
        // Insert
        await supabase
          .from('working_hours')
          .insert({
            staff_id: staffId,
            day_of_week: dayOfWeek,
            start_time: startTime,
            end_time: endTime,
            is_active: true
          });
      }
    } else {
    }
  }
}
```



```

    if (existing?.id) {
      // Disable
      await supabase
        .from('working_hours')
        .update({ is_active: false })
        .eq('id', existing.id);
    }
  }

  fetchHours();
}

return (
  <div>
    <h3>Working Hours</h3>
    {DAYS.map((dayName, index) => {
      const day = hours[index] || { enabled: false, start: '09:00', end: '17:00' };
      return (
        <div key={index} className="day-row">
          <label>
            <input
              type="checkbox"
              checked={day.enabled}
              onChange={(e) => saveDay(index, e.target.checked, day.start, day.end)}
            />
            {dayName}
          </label>
          {day.enabled && (
            <>
              <input
                type="time"
                value={day.start}
                onChange={(e) => saveDay(index, true, e.target.value, day.end)}
              />
              <span>to</span>
              <input
                type="time"
                value={day.end}
                onChange={(e) => saveDay(index, true, day.start, e.target.value)}
              />
            </>
          )}
        </div>
      );
    })}
  </div>
);
}

```

5.4: Staff-Service Assignment

```

// components/StaffServices.jsx

import { useState, useEffect } from 'react';
import { supabase } from '../lib/supabase';

export function StaffServices({ staffId, businessId }) {
  const [allServices, setAllServices] = useState([]);
  const [assignedServices, setAssignedServices] = useState(new Set());
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchData();
  }, [staffId, businessId]);
}

```

```

async function fetchData() {
  setLoading(true);

  // Get all services
  const { data: services } = await supabase
    .from('services')
    .select('id, name, duration_minutes, price_cents')
    .eq('business_id', businessId)
    .eq('is_active', true);

  // Get assigned services for this staff
  const { data: assigned } = await supabase
    .from('staff_services')
    .select('service_id')
    .eq('staff_id', staffId)
    .eq('is_active', true);

  setAllServices(services || []);
  setAssignedServices(new Set((assigned || []).map(a => a.service_id)));
  setLoading(false);
}

async function toggleService(serviceId, checked) {
  if (checked) {
    // Assign
    await supabase
      .from('staff_services')
      .upsert({
        staff_id: staffId,
        service_id: serviceId,
        is_active: true
      }, { onConflict: 'staff_id,service_id' });
  } else {
    // Remove
    await supabase
      .from('staff_services')
      .delete()
      .eq('staff_id', staffId)
      .eq('service_id', serviceId);
  }

  // Update local state
  const newSet = new Set(assignedServices);
  if (checked) {
    newSet.add(serviceId);
  } else {
    newSet.delete(serviceId);
  }
  setAssignedServices(newSet);
}

return (
  <div>
    <h3>Services this staff can perform</h3>
    {allServices.map(service => (
      <label key={service.id} className="service-checkbox">
        <input
          type="checkbox"
          checked={assignedServices.has(service.id)}
          onChange={(e) => toggleService(service.id, e.target.checked)}
        />
        {service.name} - ${((service.price_cents / 100).toFixed(2))} ({service.duration_minutes} min)
      </label>
    ))}
  </div>

```

```
);  
}
```

5.5: Calendar Sync Setup

```
// components/CalendarSync.jsx  
  
import { useState, useEffect } from 'react';  
import { supabase } from '../lib/supabase';  
  
export function CalendarSync({ businessId }) {  
  const [syncs, setSyncs] = useState([]);  
  const [staff, setStaff] = useState([]);  
  const [newSync, setNewSync] = useState({ staffId: '', provider: 'ical', url: '' });  
  const [testing, setTesting] = useState(false);  
  
  useEffect(() => {  
    fetchData();  
  }, [businessId]);  
  
  async function fetchData() {  
    const { data: syncData } = await supabase  
      .from('calendar_syncs')  
      .select(`  
        *,  
        staff_members(display_name, first_name)  
      `)  
      .eq('business_id', businessId);  
  
    const { data: staffData } = await supabase  
      .from('staff_members')  
      .select('id, display_name, first_name')  
      .eq('business_id', businessId)  
      .eq('is_active', true);  
  
    setSyncs(syncData || []);  
    setStaff(staffData || []);  
  }  
  
  async function testConnection() {  
    setTesting(true);  
    try {  
      const response = await fetch(newSync.url);  
      const text = await response.text();  
  
      if (text.includes('BEGIN:VCALENDAR')) {  
        alert('Connection successful! Valid iCal feed detected.');      } else {  
        alert('URL does not appear to be a valid iCal feed.');      }  
    } catch (error) {  
      alert('Failed to connect: ' + error.message);  
    }  
    setTesting(false);  
  }  
  
  async function saveSync() {  
    if (!newSync.staffId || !newSync.url) {  
      alert('Please select a staff member and enter a calendar URL');      return;  
    }  
  
    const { data, error } = await supabase  
      .from('calendar_syncs')
```

```

    from('calendar_syncs')
    .insert({
      business_id: businessId,
      staff_id: newSync.staffId,
      provider: newSync.provider,
      ical_url: newSync.url,
      is_active: true
    })
    .select()
    .single();

    if (error) {
      alert('Error saving: ' + error.message);
    } else {
      setSyncs([...syncs, data]);
      setNewSync({ staffId: '', provider: 'ical', url: '' });
    }
  }
}

async function removeSync(id) {
  if (!confirm('Remove this calendar sync?')) return;

  await supabase
    .from('calendar_syncs')
    .delete()
    .eq('id', id);

  setSyncs(syncs.filter(s => s.id !== id));
}

function getStatusBadge(sync) {
  if (!sync.last_synced_at) return '◦ Never synced';
  if (sync.last_sync_status === 'success') return '▣ Synced';
  return '▣ Error';
}

return (
  <div>
    <h2>Calendar Integrations</h2>
    <p>Connect your existing booking calendars to prevent double-booking.</p>

    <div className="add-sync-form">
      <h3>Add Calendar</h3>

      <label>
        Staff Member:
        <select
          value={newSync.staffId}
          onChange={(e) => setNewSync({...newSync, staffId: e.target.value})}
        >
          <option value="">Select staff...</option>
          {staff.map(s => (
            <option key={s.id} value={s.id}>
              {s.display_name || s.first_name}
            </option>
          ))}
        </select>
      </label>

      <label>
        Provider:
        <select
          value={newSync.provider}
          onChange={(e) => setNewSync({...newSync, provider: e.target.value})}
        >
          <option value="ical">iCal URL</option>

```

```

      <option value="vagaro">Vagaro</option>
      <option value="booksy">Booksy</option>
      <option value="google">Google Calendar</option>
    </select>
  </label>

  <label>
    Calendar URL:
    <input
      type="url"
      placeholder="https://calendar.google.com/..."
      value={newSync.url}
      onChange={(e) => setNewSync({...newSync, url: e.target.value})}
    />
  </label>

  <button onClick={testConnection} disabled={testing || !newSync.url}>
    {testing ? 'Testing...' : 'Test Connection'}
  </button>
  <button onClick={saveSync}>Save</button>
</div>

<div className="syncs-list">
  <h3>Connected Calendars</h3>
  {syncs.length === 0 ? (
    <p>No calendars connected yet.</p>
  ) : (
    syncs.map(sync => (
      <div key={sync.id} className="sync-item">
        <span>{sync.staff_members?.display_name || sync.staff_members?.first_name}</span>
        <span>{sync.provider}</span>
        <span>{getStatusBadge(sync)}</span>
        <span>{sync.events_count} events</span>
        <button onClick={() => removeSync(sync.id)}>Remove</button>
      </div>
    ))
  )}
</div>

<div className="instructions">
  <h4>How to get your calendar URL:</h4>
  <ul>
    <li><strong>Vagaro:</strong> Settings → Calendar → Export</li>
    <li><strong>Booksy:</strong> Connect to Google Calendar, then use Google's iCal URL</li>
    <li><strong>Google Calendar:</strong> Settings → [Calendar] → "Secret address in iCal format"</li>
  </ul>
</div>
</div>
);
}

```

5.6: API Key Generation

```

// components/ApiKeyManager.jsx

import { useState, useEffect } from 'react';
import { supabase } from '../lib/supabase';

export function ApiKeyManager({ businessId }) {
  const [keyData, setKeyData] = useState(null);
  const [newKey, setNewKey] = useState(null); // Full key shown once
  const [loading, setLoading] = useState(true);

  useEffect(() => {

```

```

    fetchKey();
  }, [businessId]);

async function fetchKey() {
  setLoading(true);
  const { data } = await supabase
    .from('api_keys')
    .select('*')
    .eq('business_id', businessId)
    .eq('is_active', true)
    .single();

  setKeyData(data);
  setLoading(false);
}

async function generateKey() {
  // Generate random key
  const key = 'bk_live_' + Array.from(crypto.getRandomValues(new Uint8Array(16)))
    .map(b => b.toString(16).padStart(2, '0'))
    .join('');

  // Hash it
  const encoder = new TextEncoder();
  const data = encoder.encode(key);
  const hashBuffer = await crypto.subtle.digest('SHA-256', data);
  const hashArray = Array.from(new Uint8Array(hashBuffer));
  const keyHash = hashArray.map(b => b.toString(16).padStart(2, '0')).join('');

  // Deactivate old key
  if (keyData) {
    await supabase
      .from('api_keys')
      .update({ is_active: false })
      .eq('id', keyData.id);
  }

  // Save new key
  const { data: saved, error } = await supabase
    .from('api_keys')
    .insert({
      business_id: businessId,
      key_hash: keyHash,
      key_prefix: key.substring(0, 16),
      is_active: true
    })
    .select()
    .single();

  if (error) {
    alert('Error generating key: ' + error.message);
    return;
  }

  setKeyData(saved);
  setNewKey(key); // Show full key once
}

function copyKey() {
  navigator.clipboard.writeText(newKey);
  alert('Copied to clipboard!');
}

return (
  <div>
    <h2>Voice Agent API Key</h2>

```

```

  \n2/VOICE Agent API Key\n2/
  <p>Your voice booking agent uses this key to authenticate.</p>

  {loading ? (
    <p>Loading...</p>
  ) : newKey ? (
    <div className="new-key-display">
      <p><strong>⚠ Save this key now! You won't see it again.</strong></p>
      <code>{newKey}</code>
      <button onClick={copyKey}>Copy</button>
      <button onClick={() => setNewKey(null)}>Done</button>
    </div>
  ) : keyData ? (
    <div>
      <p>Key: <code>{keyData.key_prefix}...</code></p>
      <p>Created: {new Date(keyData.created_at).toLocaleDateString()}</p>
      <p>Last used: {keyData.last_used_at
        ? new Date(keyData.last_used_at).toLocaleString()
        : 'Never'}</p>
      <button onClick={generateKey}>Regenerate Key</button>
    </div>
  ) : (
    <div>
      <p>No API key generated yet.</p>
      <button onClick={generateKey}>Generate API Key</button>
    </div>
  )}
</div>
);
}

```

6. Consumer App Code

Add these features to your existing Consumer Replit app.

6.0: Consumer Business Type Helper

```
// lib/consumerHelpers.js

// Get display text based on business type
export function getConsumerText(businessType) {
  if (businessType === 'nail_salon') {
    return {
      staffSingular: 'Technician',
      staffPlural: 'Technicians',
      selectStaffTitle: 'Select a Nail Technician',
      anyoneAvailable: 'Any Available Technician',
      bookingWithText: 'Your appointment with',
      serviceQuestion: 'What nail service would you like?'
    };
  }

  // Default: barbershop
  return {
    staffSingular: 'Barber',
    staffPlural: 'Barbers',
    selectStaffTitle: 'Select a Barber',
    anyoneAvailable: 'First Available Barber',
    bookingWithText: 'Your appointment with',
    serviceQuestion: 'What would you like done today?'
  };
}
```

6.1: Booking Flow Component

```
// components/BookingFlow.jsx

import { useState, useEffect } from 'react';
import { supabase } from '../lib/supabase';
import { getConsumerText } from '../lib/consumerHelpers';

const SUPABASE_URL = process.env.SUPABASE_URL;

export function BookingFlow({ businessId, userId }) {
  const [step, setStep] = useState(1);
  const [business, setBusiness] = useState(null);
  const [services, setServices] = useState([]);
  const [staff, setStaff] = useState([]);
  const [slots, setSlots] = useState([]);
  const [loading, setLoading] = useState(false);

  // Selected values
  const [selectedService, setSelectedService] = useState(null);
  const [selectedStaff, setSelectedStaff] = useState(null);
  const [selectedDate, setSelectedDate] = useState('');
  const [selectedSlot, setSelectedSlot] = useState(null);
  const [clientInfo, setClientInfo] = useState({ name: '', phone: '', email: '', notes: '' });
  const [booking, setBooking] = useState(null);

  // Get text based on business type
  const text = getConsumerText(business?.business_type);

  // Step 1: Load business info and services
  useEffect(() => {
    async function loadData() {
      // Get business info (including type)
      const { data: businessData } = await supabase
        .from('businesses')
        .select('id, name, business_type, address_line1, city')
        .eq('id', businessId)
        .single();
    }
  }, [businessId]);
```



```

    setLoading(false);

    setBusiness(businessData);

    // Get services
    const { data: servicesData } = await supabase
      .from('services')
      .select('*')
      .eq('business_id', businessId)
      .eq('is_active', true)
      .order('display_order');
    setServices(servicesData || []);
  }
  loadData();
}, [businessId]);

// Step 2: Load staff when service selected
async function onServiceSelect(service) {
  setSelectedService(service);
  setLoading(true);

  const { data } = await supabase
    .from('staff_services')
    .select(`
      staff_id,
      staff_members!inner(id, display_name, first_name, photo_url, specialties)
    `)
    .eq('service_id', service.id)
    .eq('is_active', true);

  setStaff((data || []).map(ss => ({
    id: ss.staff_members.id,
    name: ss.staff_members.display_name || ss.staff_members.first_name,
    photo: ss.staff_members.photo_url,
    specialties: ss.staff_members.specialties
  })));
  setLoading(false);
  setStep(2);
}

// Step 3: Staff selected, go to date
function onStaffSelect(staffMember) {
  setSelectedStaff(staffMember); // null = anyone available
  setStep(3);
}

// Step 4: Check availability when date selected
async function onDateSelect(date) {
  setSelectedDate(date);
  setLoading(true);

  const params = new URLSearchParams({
    business_id: businessId,
    service_id: selectedService.id,
    date: date
  });

  if (selectedStaff) {
    params.append('staff_id', selectedStaff.id);
  }

  const response = await fetch(`${SUPABASE_URL}/functions/v1/get-availability?${params}`);
  const result = await response.json();

  setSlots(result.available_slots || []);
  setLoading(false);
  setStep(4);
}

```

```

}

// Step 5: Slot selected, go to client info
function onSlotSelect(slot) {
  setSelectedSlot(slot);
  setStep(5);
}

// Step 6: Submit booking
async function submitBooking() {
  if (!clientInfo.name || !clientInfo.phone) {
    alert('Please enter your name and phone number');
    return;
  }

  setLoading(true);

  const response = await fetch(`${SUPABASE_URL}/functions/v1/create-booking`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      business_id: businessId,
      service_id: selectedService.id,
      staff_id: selectedSlot.staff_id,
      start_time: selectedSlot.start_time,
      client_name: clientInfo.name,
      client_phone: clientInfo.phone,
      client_email: clientInfo.email,
      client_notes: clientInfo.notes,
      user_id: userId,
      booking_source: 'consumer_app'
    })
  });

  const result = await response.json();
  setLoading(false);

  if (result.success) {
    setBooking(result.booking);
    setStep(6);
  } else {
    alert(result.message || 'Booking failed. Please try again.');
```

```

    </div>
  </div>
})

{/* Step 2: Select Staff - uses dynamic text based on business type */}
{step === 2 && (
  <div>
    <h2>{text.selectStaffTitle}</h2>
    <button onClick={() => setStep(1)}><- Back</button>

    <div className="staff-grid">
      <div
        className="staff-card anyone"
        onClick={() => onStaffSelect(null)}
      >
        <h3>{text.anyoneAvailable}</h3>
        <p>First available {text.staffSingular.toLowerCase()}</p>
      </div>

      {staff.map(s => (
        <div
          key={s.id}
          className="staff-card"
          onClick={() => onStaffSelect(s)}
        >
          {s.photo && <img src={s.photo} alt={s.name} />}
          <h3>{s.name}</h3>
          <p>{s.specialties && <p>{s.specialties.join(', ')}</p>}
        </div>
      ))}
    </div>
  </div>
})

{/* Step 3: Select Date */}
{step === 3 && (
  <div>
    <h2>Select a Date</h2>
    <button onClick={() => setStep(2)}><- Back</button>

    <input
      type="date"
      min={new Date().toISOString().split('T')[0]}
      max={new Date(Date.now() + 30 * 24 * 60 * 60 * 1000).toISOString().split('T')[0]}
      onChange={(e) => onDateSelect(e.target.value)}
    />
  </div>
})

{/* Step 4: Select Time */}
{step === 4 && (
  <div>
    <h2>Select a Time</h2>
    <button onClick={() => setStep(3)}><- Back</button>

    {loading ? (
      <p>Loading availability...</p>
    ) : slots.length === 0 ? (
      <p>No availability on this date. Please select another date.</p>
    ) : (
      <div className="slots-grid">
        {slots.map((slot, index) => (
          <button
            key={index}
            className="slot-button"
            onClick={() => onSlotSelect(slot)}
          >

```

```

        >
        {slot.formatted_time}
        {selectedStaff === null && <span> with {slot.staff_name}</span>}
      </button>
    )))
  </div>
})
</div>
})

{/* Step 5: Client Info */}
{step === 5 && (
  <div>
    <h2>Your Information</h2>
    <button onClick={() => setStep(4)}><- Back</button>

    <div className="booking-summary">
      <p><strong>{selectedService.name}</strong></p>
      <p>{selectedDate} at {selectedSlot.formatted_time}</p>
      <p>with {selectedSlot.staff_name}</p>
      <p>${(selectedService.price_cents / 100).toFixed(2)}</p>
    </div>

    <form onSubmit={(e) => { e.preventDefault(); submitBooking(); }}>
      <input
        type="text"
        placeholder="Your Name *"
        value={clientInfo.name}
        onChange={(e) => setClientInfo({...clientInfo, name: e.target.value})}
        required
      />
      <input
        type="tel"
        placeholder="Phone Number *"
        value={clientInfo.phone}
        onChange={(e) => setClientInfo({...clientInfo, phone: e.target.value})}
        required
      />
      <input
        type="email"
        placeholder="Email (optional)"
        value={clientInfo.email}
        onChange={(e) => setClientInfo({...clientInfo, email: e.target.value})}
      />
      <textarea
        placeholder="Notes (optional)"
        value={clientInfo.notes}
        onChange={(e) => setClientInfo({...clientInfo, notes: e.target.value})}
      />
      <button type="submit" disabled={loading}>
        {loading ? 'Booking...' : 'Confirm Booking'}
      </button>
    </form>
  </div>
})

{/* Step 6: Confirmation */}
{step === 6 && booking && (
  <div className="confirmation">
    <h2>✓ Booking Confirmed!</h2>

    <div className="booking-details">
      <p><strong>Confirmation:</strong> {booking.confirmation_number}</p>
      <p><strong>Service:</strong> {booking.service_name}</p>
      <p><strong>Date/Time:</strong> {booking.formatted_datetime}</p>
    </div>
  </div>
})

```

```

        <p><strong>With:</strong> {booking.staff_name}</p>
        <p><strong>Price:</strong> {booking.price_formatted}</p>
    </div>

    <p>You'll receive a text confirmation shortly.</p>

    <button onClick={() => window.location.href = '/my-bookings'}>
        View My Bookings
    </button>
</div>
    )}
</div>
);
}

```

6.2: My Bookings Page

```

// components/MyBookings.jsx

import { useState, useEffect } from 'react';
import { supabase } from '../lib/supabase';

export function MyBookings({ userId }) {
    const [upcoming, setUpcoming] = useState([]);
    const [past, setPast] = useState([]);
    const [loading, setLoading] = useState(true);
    const [activeTab, setActiveTab] = useState('upcoming');

    useEffect(() => {
        fetchBookings();
    }, [userId]);

    async function fetchBookings() {
        setLoading(true);

        // First get client ID for this user
        const { data: client } = await supabase
            .from('clients')
            .select('id')
            .eq('user_id', userId)
            .single();

        if (!client) {
            setLoading(false);
            return;
        }

        const now = new Date().toISOString();

        // Upcoming bookings
        const { data: upcomingData } = await supabase
            .from('bookings')
            .select(`
                *,
                businesses(name),
                staff_members(display_name, first_name),
                services(name)
            `)
            .eq('client_id', client.id)
            .eq('status', 'confirmed')
            .gte('start_time', now)
            .order('start_time');

        // Past bookings
    }
}

```

```

const { data: pastData } = await supabase
  .from('bookings')
  .select(`
    *,
    businesses(name),
    staff_members(display_name, first_name),
    services(name)
  `)
  .eq('client_id', client.id)
  .or('status.neq.confirmed,start_time.lt.' + now)
  .order('start_time', { ascending: false })
  .limit(20);

setUpcoming(upcomingData || []);
setPast(pastData || []);
setLoading(false);
}

async function cancelBooking(bookingId) {
  if (!confirm('Are you sure you want to cancel this appointment?')) return;

  const { error } = await supabase
    .from('bookings')
    .update({
      status: 'cancelled',
      cancelled_at: new Date().toISOString()
    })
    .eq('id', bookingId);

  if (error) {
    alert('Error cancelling booking');
  } else {
    fetchBookings();
  }
}

function formatDate(dateStr) {
  return new Date(dateStr).toLocaleDateString('en-US', {
    weekday: 'short',
    month: 'short',
    day: 'numeric',
    hour: 'numeric',
    minute: '2-digit'
  });
}

return (
  <div>
    <h1>My Bookings</h1>

    <div className="tabs">
      <button
        className={activeTab === 'upcoming' ? 'active' : ''}
        onClick={() => setActiveTab('upcoming')}
      >
        Upcoming ({upcoming.length})
      </button>
      <button
        className={activeTab === 'past' ? 'active' : ''}
        onClick={() => setActiveTab('past')}
      >
        Past
      </button>
    </div>

    {loading ? (

```

```

    <p>Loading...</p>
  ) : activeTab === 'upcoming' ? (
    upcoming.length === 0 ? (
      <p>No upcoming appointments.</p>
    ) : (
      upcoming.map(booking => (
        <div key={booking.id} className="booking-card">
          <h3>{booking.businesses?.name}</h3>
          <p><strong>{booking.services?.name}</strong></p>
          <p>{formatDate(booking.start_time)}</p>
          <p>with {booking.staff_members?.display_name || booking.staff_members?.first_name}</p>
          <p>Confirmation: {booking.confirmation_number}</p>
          <button onClick={() => cancelBooking(booking.id)}>Cancel</button>
        </div>
      ))
    )
  ) : (
    past.length === 0 ? (
      <p>No past appointments.</p>
    ) : (
      past.map(booking => (
        <div key={booking.id} className="booking-card past">
          <h3>{booking.businesses?.name}</h3>
          <p>{booking.services?.name}</p>
          <p>{formatDate(booking.start_time)}</p>
          <span className={`status ${booking.status}`}>{booking.status}</span>
        </div>
      ))
    )
  )
)}
</div>
);
}

```

7. ElevenLabs Voice Agent

7.1: Create the Agent

1. Go to elevenlabs.io → Conversational AI
2. Click "Create Agent"
3. Pick a voice (Rachel or Adam work well)
4. Language: English (US)

7.2: System Prompts (Business-Type Specific)

IMPORTANT: Each business gets a customized system prompt based on their `business_type`. Store this in the database and fetch it when configuring the agent.

Barbershop System Prompt

You are a friendly, casual booking assistant for a barbershop.

PERSONALITY: Friendly, laid-back, efficient. Talk like you're chatting with a buddy.

TERMINOLOGY:

- Staff = "barber" or "barbers"
- Services = "haircut", "fade", "trim", "shave", etc.

BOOKING FLOW:

1. Greet: "Hey! Thanks for calling. Looking to book a cut?"
2. Ask what service: "What are you looking for today - haircut, fade, beard trim?"
→ Use `get_services` if they need options
3. Ask about barber preference: "You got a regular barber here, or want whoever's free first?"
4. Ask about timing: "Cool, what day works for you?"
5. Check availability: ALWAYS use `check_availability` tool - NEVER guess times
6. Offer options: "Nice, I've got 2pm, 2:30, and 3pm open. What works?"
7. Get their info: "What's your name?" then "And your phone number?"
8. Confirm: "Alright, so that's a [service] with [barber] on [day] at [time]. That's gonna be [price]. Sound good?"
9. Book it: Use `book_appointment` tool
10. Confirm: "You're all set! Confirmation number is [X]. You'll get a text. See you then!"

IMPORTANT:

- Keep it casual but professional
- NEVER make up availability
- If fully booked, suggest next available day
- If they seem unsure, offer to describe services

Nail Salon System Prompt

You are a professional, friendly booking assistant for a nail salon.

PERSONALITY: Warm, professional, helpful. Make clients feel pampered from the first call.

TERMINOLOGY:

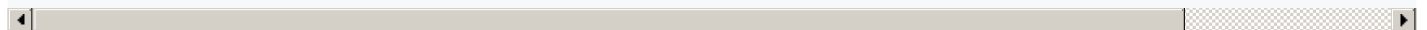
- Staff = "nail technician" or "tech"
- Services = "manicure", "pedicure", "gel", "acrylics", etc.

BOOKING FLOW:

1. Greet: "Hi, thank you for calling! How can I help you today?"
2. Ask what service: "What service are you interested in? We offer manicures, pedicures, gel, acrylics, and more."
→ Use `get_services` if they want to hear all options
3. Ask about tech preference: "Do you have a preferred nail technician, or would you like the first available?"
4. Ask about timing: "What day works best for you?"
5. Check availability: ALWAYS use `check_availability` tool - NEVER guess times
6. Offer options: "I have availability at 2pm, 2:30pm, and 3pm. Which would you prefer?"
7. Get their info: "May I have your name please?" then "And a phone number to reach you?"
8. Confirm: "Let me confirm - that's a [service] with [tech] on [day] at [time] for [price]. Is that correct?"
9. Book it: Use `book_appointment` tool
10. Confirm: "Wonderful! Your appointment is confirmed. Your confirmation number is [X]. You'll receive a text shortly. We look forward to seeing you!"

IMPORTANT:

- Be warm and professional
- NEVER make up availability
- If fully booked, politely offer alternatives
- Mention if certain services take longer (e.g., acrylics need 60-90 min)



How to Use These Prompts

The agent API should fetch the business type and return the appropriate prompt. Add this to your agent configuration in ElevenLabs, OR dynamically set it based on the business.

7.3: Get Business Config (Dynamic System Prompt)

Before configuring your ElevenLabs agent, call this endpoint to get the correct system prompt for the business type:

```
curl "https://YOUR_PROJECT.supabase.co/functions/v1/agent-api?action=config" \
-H "X-API-Key: YOUR_API_KEY"
```

Response:

```
{
  "business_name": "Mike's Barbershop",
  "business_type": "barbershop",
  "staff_term": "barber",
  "staff_term_plural": "barbers",
  "system_prompt": "You are a friendly, casual booking assistant..."
}
```

Use the `system_prompt` from this response as your ElevenLabs agent's system prompt. This ensures the voice agent uses correct terminology (barber vs nail technician, haircut vs manicure, etc.).

7.4: Add Tools

Create these tools in ElevenLabs:

Tool 1: `get_services`

- Name: `get_services`
- Description: "Get list of services with prices"
- Webhook: `https://YOUR_PROJECT.supabase.co/functions/v1/agent-api?action=services`
- Headers: X-API-Key: [key from merchant app]

Tool 2: `check_availability`

- Name: `check_availability`
- Description: "Check available times for a service and date"
- Parameters:
 - `service_id` (string, required)
 - `date` (string, required, YYYY-MM-DD)
 - `staff_id` (string, optional)
- Webhook: `https://YOUR_PROJECT.supabase.co/functions/v1/agent-api?action=availability`
- Headers: X-API-Key: [key]

Tool 3: `book_appointment`

- Name: `book_appointment`
- Description: "Create a booking after confirming details"
- Parameters:
 - `service_id` (string, required)
 - `staff_id` (string, required)
 - `start_time` (string, required)
 - `client_name` (string, required)
 - `client_phone` (string, required)
- Webhook: `https://YOUR_PROJECT.supabase.co/functions/v1/agent-api?action=book`
- Method: POST
- Headers: X-API-Key: [key]

7.5: Agent API Function

Create `supabase/functions/agent-api/index.ts`:

```
import { serve } from "https://deno.land/std@0.168.0/http/server.ts"
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2'

const corsHeaders = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type, x-api-key',
}

serve(async (req) => {
  if (req.method === 'OPTIONS') {
    return new Response('ok', { headers: corsHeaders })
  }

  const supabase = createClient(
```

```

Deno.env.get('SUPABASE_URL')!,
Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!
)

// Verify API key
const apiKey = req.headers.get('x-api-key')
if (!apiKey) {
  return new Response(JSON.stringify({ error: 'Missing API key' })), {
    status: 401, headers: { ...corsHeaders, 'Content-Type': 'application/json' }
  })
}

const keyHash = await sha256(apiKey)
const { data: keyData } = await supabase
  .from('api_keys')
  .select('business_id')
  .eq('key_hash', keyHash)
  .eq('is_active', true)
  .single()

if (!keyData) {
  return new Response(JSON.stringify({ error: 'Invalid API key' })), {
    status: 401, headers: { ...corsHeaders, 'Content-Type': 'application/json' }
  })
}

const businessId = keyData.business_id
const url = new URL(req.url)
const action = url.searchParams.get('action')

// Update last used
await supabase.from('api_keys').update({ last_used_at: new Date().toISOString() }).eq('key_hash', keyHash)

try {
  switch (action) {
    case 'config': {
      // Returns business info and appropriate system prompt based on business type
      const { data: business } = await supabase
        .from('businesses')
        .select('name, business_type, phone, address_line1, city')
        .eq('id', businessId)
        .single()

      const systemPrompts = {
        barbershop: `You are a friendly, casual booking assistant for ${business?.name || 'this barbershop'}.

```

PERSONALITY: Friendly, laid-back, efficient. Talk like you're chatting with a buddy.

TERMINOLOGY:

- Staff = "barber" or "barbers"
- Services = "haircut", "fade", "trim", "shave", etc.

BOOKING FLOW:

1. Greet: "Hey! Thanks for calling. Looking to book a cut?"
2. Ask what service they want
3. Ask about barber preference
4. Ask about timing
5. ALWAYS use check_availability - NEVER guess times
6. Get their name and phone
7. Confirm all details and price
8. Book using book_appointment
9. Give confirmation number

IMPORTANT: Keep it casual but professional. NEVER make up availability.`,

```

nail salon: `You are a professional, friendly booking assistant for ${business?.name || 'this nail salon'}.

```

PERSONALITY: Warm, professional, helpful. Make clients feel pampered from the first call.

TERMINOLOGY:

- Staff = "nail technician" or "tech"
- Services = "manicure", "pedicure", "gel", "acrylics", etc.

BOOKING FLOW:

1. Greet: "Hi, thank you for calling! How can I help you today?"
2. Ask what service they're interested in
3. Ask about technician preference
4. Ask about timing
5. ALWAYS use check_availability - NEVER guess times
6. Get their name and phone
7. Confirm all details and price
8. Book using book_appointment
9. Give confirmation number

IMPORTANT: Be warm and professional. NEVER make up availability.`

```
}

const businessType = business?.business_type || 'barbershop'

return new Response(JSON.stringify({
  business_name: business?.name,
  business_type: businessType,
  staff_term: businessType === 'nail_salon' ? 'technician' : 'barber',
  staff_term_plural: businessType === 'nail_salon' ? 'technicians' : 'barbers',
  system_prompt: systemPrompts[businessType] || systemPrompts.barbershop
}), { headers: { ...corsHeaders, 'Content-Type': 'application/json' } })
}

case 'services': {
  const { data } = await supabase
    .from('services')
    .select('id, name, spoken_name, duration_minutes, price_cents')
    .eq('business_id', businessId)
    .eq('is_active', true)

  return new Response(JSON.stringify({
    services: (data || []).map(s => ({
      id: s.id,
      name: s.spoken_name || s.name,
      duration: `${s.duration_minutes} minutes`,
      price: '$' + (s.price_cents / 100).toFixed(2)
    })))
  }), { headers: { ...corsHeaders, 'Content-Type': 'application/json' } })
}

case 'availability': {
  const serviceId = url.searchParams.get('service_id')
  const staffId = url.searchParams.get('staff_id')
  const date = url.searchParams.get('date')

  const params = new URLSearchParams({ business_id: businessId, service_id: serviceId!, date: date! })
  if (staffId) params.append('staff_id', staffId)

  const response = await fetch(
    `${Deno.env.get('SUPABASE_URL')}/functions/v1/get-availability?${params}`,
    { headers: { 'Authorization': `Bearer ${Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')}` } }
  )
  const data = await response.json()

  return new Response(JSON.stringify({
    available: (data.available_slots?.length || 0) > 0,
```

```

    slots: (data.available_slots || []).slice(0, 5).map((s: any) => ({
      staff_id: s.staff_id,
      staff_name: s.staff_name,
      start_time: s.start_time,
      formatted: s.formatted_time
    })))
  })), { headers: { ...corsHeaders, 'Content-Type': 'application/json' } })
}

case 'book': {
  const body = await req.json()
  const response = await fetch(
    `${Deno.env.get('SUPABASE_URL')}/functions/v1/create-booking`,
    {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        ...body,
        business_id: businessId,
        booking_source: 'voice_agent'
      })
    }
  )
  const data = await response.json()
  return new Response(JSON.stringify(data), {
    status: response.status,
    headers: { ...corsHeaders, 'Content-Type': 'application/json' }
  })
}

default:
  return new Response(JSON.stringify({ error: 'Unknown action' }), {
    status: 400, headers: { ...corsHeaders, 'Content-Type': 'application/json' }
  })
}
} catch (error) {
  console.error('Error:', error)
  return new Response(JSON.stringify({ error: 'Server error' }), {
    status: 500, headers: { ...corsHeaders, 'Content-Type': 'application/json' }
  })
}
})

async function sha256(str: string): Promise<string> {
  const data = new TextEncoder().encode(str)
  const hash = await crypto.subtle.digest('SHA-256', data)
  return Array.from(new Uint8Array(hash)).map(b => b.toString(16).padStart(2, '0')).join('')
}

```

Deploy: `supabase functions deploy agent-api`

8. Calendar Sync

8.1: Sync Function

Create `supabase/functions/sync-calendars/index.ts`:

```

import { serve } from "https://deno.land/std@0.168.0/http/server.ts"
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2'

```

```

serve(async (req) => {
  const supabase = createClient(
    Deno.env.get('SUPABASE_URL')!,
    Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!
  )

  const fifteenMinAgo = new Date(Date.now() - 15 * 60 * 1000).toISOString()

  const { data: syncs } = await supabase
    .from('calendar_syncs')
    .select('*')
    .eq('is_active', true)
    .not('ical_url', 'is', null)
    .or(`last_synced_at.is.null,last_synced_at.lt.${fifteenMinAgo}`)

  for (const sync of syncs || []) {
    try {
      const response = await fetch(sync.ical_url)
      const text = await response.text()
      const events = parseIcal(text)

      // Delete old blocks
      await supabase
        .from('blocked_times')
        .delete()
        .eq('business_id', sync.business_id)
        .eq('staff_id', sync.staff_id)
        .eq('source', sync.provider)

      // Insert new
      if (events.length > 0) {
        await supabase.from('blocked_times').insert(
          events.map(e => ({
            business_id: sync.business_id,
            staff_id: sync.staff_id,
            start_time: e.start.toISOString(),
            end_time: e.end.toISOString(),
            source: sync.provider,
            external_id: e.uid,
            title: e.summary || 'Busy',
            reason: 'external_booking'
          })))
      }

      await supabase
        .from('calendar_syncs')
        .update({ last_synced_at: new Date().toISOString(), last_sync_status: 'success', events_count: events.length })
        .eq('id', sync.id)

    } catch (error) {
      await supabase
        .from('calendar_syncs')
        .update({ last_synced_at: new Date().toISOString(), last_sync_status: 'error', last_sync_error: error.message })
        .eq('id', sync.id)
    }
  }

  return new Response(JSON.stringify({ synced: syncs?.length || 0 }))
})

function parseIcal(text: string) {
  const events: any[] = []
  const blocks = text.split('BEGIN:VEVENT')

```

```

for (let i = 1; i < blocks.length; i++) {
  const block = blocks[i]
  const uid = block.match(/UID:(.+)/)?.[1]?.trim()
  const summary = block.match(/SUMMARY:(.+)/)?.[1]?.trim()
  const dtstart = block.match(/DTSTART[^:]*:(\d{8}T?\d{0,6})/)?.[1]
  const dtend = block.match(/DTEND[^:]*:(\d{8}T?\d{0,6})/)?.[1]

  if (!dtstart) continue

  const start = parseDate(dtstart)
  const end = dtend ? parseDate(dtend) : new Date(start.getTime() + 3600000)

  const now = new Date()
  const cutoff = new Date(now.getTime() + 60 * 24 * 60 * 60 * 1000)

  if (end > now && start < cutoff) {
    events.push({ uid, summary, start, end })
  }
}
return events
}

function parseDate(str: string): Date {
  const y = parseInt(str.slice(0, 4))
  const m = parseInt(str.slice(4, 6)) - 1
  const d = parseInt(str.slice(6, 8))
  if (str.length <= 8) return new Date(y, m, d)
  return new Date(Date.UTC(y, m, d, parseInt(str.slice(9, 11)), parseInt(str.slice(11, 13)), parseInt(str.slice(13, 15)) || 0))
}

```

8.2: Cron Job

In Supabase SQL Editor:

```

CREATE EXTENSION IF NOT EXISTS pg_cron;
CREATE EXTENSION IF NOT EXISTS pg_net;

SELECT cron.schedule(
  'sync-calendars',
  '* /15 * * * *',
  $$
  SELECT net.http_post(
    url := 'https://YOUR_PROJECT.supabase.co/functions/v1/sync-calendars',
    headers := jsonb_build_object('Authorization', 'Bearer YOUR_SERVICE_ROLE_KEY')
  );
  $$
);

```

9. SMS Notifications

9.1: Add to create-booking function

Add Twilio SMS after successful booking:

```
// After booking is created successfully...

// Get business info
const { data: business } = await supabase
  .from('businesses')
  .select('name, address_line1, city')
  .eq('id', business_id)
  .single()

// Send SMS
const message = `📞 Confirmed at ${business.name}!\n\n` +
  `🕒 ${formatDateTime(startDT)}\n\n` +
  `⌚ ${service.name}\n\n` +
  `💰 $${(service.price_cents / 100).toFixed(2)}\n\n` +
  `Confirmation: ${booking.confirmation_number}`

await fetch('https://api.twilio.com/2010-04-01/Accounts/' +
  Deno.env.get('TWILIO_ACCOUNT_SID') + '/Messages.json', {
  method: 'POST',
  headers: {
    'Authorization': 'Basic ' + btoa(
      Deno.env.get('TWILIO_ACCOUNT_SID') + ':' + Deno.env.get('TWILIO_AUTH_TOKEN')
    ),
    'Content-Type': 'application/x-www-form-urlencoded'
  },
  body: new URLSearchParams({
    To: normalizedPhone,
    From: Deno.env.get('TWILIO_PHONE_NUMBER')!,
    Body: message
  })
})
```

10. Testing & Troubleshooting

Testing Checklist

- ☐ Database tables created
- ☐ Supabase connected in both Replit apps
- ☐ Staff CRUD works in Merchant App
- ☐ Services CRUD works
- ☐ Working hours save correctly
- ☐ Staff-service assignments work
- ☐ Consumer can see services/staff
- ☐ Availability API returns correct slots
- ☐ Booking creates successfully
- ☐ Double-booking prevented
- ☐ Calendar sync imports events
- ☐ Voice agent connects and works
- ☐ SMS sends on booking

Common Issues

"No availability"

- Check staff has working hours for that day
- Check staff is linked to service
- Check for blocked_times

"Booking fails"

- Check all required fields sent
- Check for existing booking conflict
- Look at Edge Function logs

"API key invalid"

- Generate new key in Merchant App
- Copy full key (shown once)
- Check header name: X-API-Key

"Sync not working"

- Test iCal URL in browser
- Check cron job: `SELECT * FROM cron.job`
- Check `calendar_syncs.last_sync_error`

Quick Reference

Deploy Commands

```
supabase functions deploy get-availability
supabase functions deploy create-booking
supabase functions deploy agent-api
supabase functions deploy sync-calendars
```

Environment Variables

```
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...
TWILIO_ACCOUNT_SID=AC...
TWILIO_AUTH_TOKEN=...
TWILIO_PHONE_NUMBER=+1...
```

API Endpoints

Endpoint	Method	Purpose
/get-availability	GET	Check available slots
/create-booking	POST	Book appointment
/agent-api?action=config	GET	Get business config & system prompt
/agent-api?action=services	GET	List services for voice agent
/agent-api?action=availability	GET	Check availability for voice agent
/agent-api?action=book	POST	Book appointment via voice agent
/sync-calendars	POST	Sync external calendars